

Lighting Match Engine Core

One-page repo summary generated from code and docs in this workspace only.

What It Is

A Rust matching-engine core for a single product that focuses on low-latency in-memory order matching. The current repo behaves like a benchmark/demo executable that exercises auctions, continuous trading, and performance reporting.

Who It's For

Primary persona: engineers building or evaluating low-latency exchange/trading infrastructure who want a compact matching core they can embed inside a larger stack.

What It Does

- Two order book implementations: **dense** and **sparse**, selected by CLI/config.
- Supports **limit** and **market** price types in shared order data structures.
- Implements **continuous matching** plus opening/closing **call auction** flows.
- Validates dense-book prices for configured **tick** and supported range.
- Encodes submit/cancel/trade/stats/error messages into fixed **64-byte** wire packets.
- Includes built-in **performance timing**, result tables, and latency statistics output.
- Can pin the matching thread to a CPU core via the cross-platform **cpu_affinity** module.

How It Works

- CLI flags and a few env vars feed **config::get_config()**, producing **AppConfig** with instance, product, and order-book settings.
- **orderbook::factory::build_order_book()** creates either a dense or sparse in-memory book.
- **EngineState** is the runtime coordinator: it tracks market phase, routes orders to the active call-auction pool or the continuous order book, and stores match counts/outcomes.
- **SessionRunner** drives a demo session through opening auction, continuous trading, and optional closing auction based on market structure.
- **protocol::codec** serializes orders, cancels, trades, stats, and reject replies; live network I/O/listeners are **Not found in repo**.
- Main data flow in **src/main.rs**: parse config -> build book -> create engine state -> run demo session -> seed test book -> submit benchmark orders -> print stats / serialize replies on errors.

How To Run

- Prereq: Rust toolchain version is shown in the repo badge as **1.70.0**; exact install steps are **Not found in repo**.
- From repo root, run `make` for the default demo benchmark.
- Minimal direct command: `cargo run --features match-timing --release -- --prodid 7 --name AAPL --test-order-book-size 50k`.
- Optional tuning flags in repo code: `--order-book dense|sparse --tick N --base-price N --max-levels N --trade-cap N`.

Repo note: README mentions UDP multicast and broadcasting, but executable socket-handling code was not located in this repo snapshot.